CIS 4020 · DATA STRUCTURES

Binary Trees and Binary Search Trees: Overview and Objectives

Tree terminology, binary tree shapes, the BST, and traversals

1. Overview

Every data structure we have built so far has been linear. Lists, stacks, and queues are linear and position oriented: you reach an item by where it sits. A sorted list is linear and value oriented: items are placed by their value, but you still walk the list one item at a time to find one. A **tree** is the first **nonlinear**, value oriented structure in the course. Because a node can have more than one item directly below it, a well shaped tree can find an item while looking at only a small fraction of the others.

This unit starts with the vocabulary of trees in general (root, leaf, parent, child, sibling, ancestor, descendant, path, subtree, level, and height) and then narrows to the **binary tree**, where every node has at most two children, a left child and a right child. We look at the shapes that have names (full, complete, and balanced) and at how a binary tree is stored in code, with each node holding an item and a reference to each child.

Most of the unit is about the **binary search tree** (BST), a binary tree that keeps its items in order by a search key. You will learn to search, insert, and delete in a BST by hand and in code, including the three cases of deletion and the inorder successor. You will also learn the four standard **traversals**: preorder, inorder, postorder, and breadth-first.

Two earlier units come back here. **Recursion** is the natural way to work with a tree, because every subtree is itself a smaller tree, and almost every method in this unit is recursive. **Algorithm analysis** explains why the shape of the tree matters so much: a BST is fast when it is short and no better than a linked list when it is tall. Stacks and queues also return, as the tools that let us traverse a tree without recursion.

This unit does not cover keeping a tree balanced. That is the subject of the next unit, balanced trees, and the performance problems you see here are the reason it exists.

As in earlier units, we work in **two languages at once**. Computer Science students write **C/C++**; CIS students write **Java**. The tree algorithms are the same in both languages. The main difference is memory: Java reclaims removed nodes for you, while C++ code must free every node it removes.

The most useful preparation is to draw. Insert a short list of numbers into a BST on paper, delete a few, and write out each traversal before you look at any code. If you can predict the tree after every step, the code is much easier to write and to debug.

2. Learning objectives

By the end of this unit you should be able to:

Remember and understand

- Explain how a tree differs from the linear structures studied earlier, and describe trees as non-linear and value oriented.
- Define the terms node, edge, root, leaf, interior node, parent, child, sibling, ancestor, descendant, path, subtree, level, and height, and identify each in a drawn tree.
- State the height convention used in this course (counting nodes, so a single node has height 1), and recognize that some sources count edges instead.
- Distinguish a general tree from a binary tree, and define left child, right child, left subtree, and right subtree.
- Define full, complete, and balanced (height balanced) binary trees, and recognize each from a drawing.
- State the binary search tree property: at every node, every key in the left subtree is less than the node's key and every key in the right subtree is greater.
- Describe how a binary tree is represented with references (Java) or pointers (C++), and how a general tree can be represented with an array or list of child references.
- Describe the order of steps in the preorder, inorder, postorder, and breadth-first traversals.
- Name the varieties of trees that come later, including the balanced trees covered in the next unit.

Apply

- Build a BST by hand by inserting a sequence of keys in a given order, and draw the tree after each insertion.
- Search a BST by hand for a key, listing the nodes visited, and identify where a missing key would be inserted.
- Delete a key from a BST by hand in each of the three cases: a leaf, a node with one child, and a node with two children.
- Find the inorder successor of a node, and use it to delete a node with two children.
- Write out the visit order of the preorder, inorder, postorder, and breadth-first traversals of a given tree.
- Trace an inorder traversal that uses a stack and a breadth-first traversal that uses a queue, showing the contents of the stack or queue at each step.
- Write recursive methods for the BST operations and the depth-first traversals in Java (CIS) or C/C++ (CS), and write the stack-based and queue-based traversals without recursion.
- Use the "return the new subtree root" pattern so that a recursive insert or delete attaches its result to the correct parent link.

- Compute the minimum and maximum possible height of a binary tree with n nodes, and the number of nodes in a full binary tree of height h.

Analyze

- Explain why search, insert, and delete in a BST each take time proportional to the height of the tree, and state the best case, $O(\log_2 n)$, and the worst case, $O(n)$.
- Explain how the order in which keys are inserted determines the shape of a BST, and show why inserting keys in sorted order produces a tree that behaves like a linked list.
- Explain why an inorder traversal of a BST visits the keys in sorted order, using the BST property.
- Explain why the inorder successor is the leftmost node of the right subtree, why copying it into the deleted node keeps the BST property, and why removing it is always a leaf case or a one-child case.
- Explain why every traversal is $O(n)$, and compare recursive traversals with their stack-based and queue-based versions.
- Determine whether a tree is a BST, and explain why checking only each node's children is not enough.
- Determine whether a tree is balanced, and explain why checking only the root is not enough.
- Relate the recursive structure of a tree to the base case and recursive step from the recursion unit, and identify the base case in each BST method.
- Explain the practical cost of a tall tree for recursive code, including running out of call stack on a very deep tree.

Evaluate

- Choose between a sorted list and a BST for a described situation, and justify the choice in terms of the operations required and their running times.
- Choose the traversal that fits a task (for example, copying a tree, freeing a tree, printing keys in order, or processing a tree level by level), and justify the choice.
- Judge whether a given BST implementation is correct, and identify and explain any error, such as a recursive call whose result is never stored in the parent link, a successor taken from the wrong subtree, or, in C++, a removed node that is never freed.
- Judge whether a plain BST is good enough for an expected pattern of input, and explain when a balanced tree is needed instead.